

GetYourGuide Senior Software Engineer — Mülakat Hazırlık Rehberi

Amaç: Bu doküman, GetYourGuide (GYG) Senior Software Engineer (özellikle Java/Backend) mülakatına **yalnızca bu guide ile** hazırlanabileceğin kadar derin bir derlemedir.

Derleme tarihi: 2026-10-07

Kapsam: Senior + Mid + Junior/Associate süreçleri, teknik + system design + behavioral, arkadaş notları, kamu kaynakları.

0. Nasıl okumalısın?

- Önce **§1 Meta-Analiz** — şirketin neye önem verdiğini anla.
- Sonra **§2 Süreç haritası** — senin loop'unun hangi varyantı olduğunu netleştir (recruiter'a sor).
- §3-§7** ile her aşamayı drill et.
- §8 Soru bankası** ve **§9 System Design** ile pratik yap.
- §10 Guiding Principles / Behavioral** için STAR hikayelerini yaz.
- §14 Kaynaklar** — her iddianın linki burada; git bak, doğrula.

Güven seviyeleri (tüm dokümanda kullanılır):

Seviye	Anlamı
A — Yüksek	Resmi GYG blog/careers, birden fazla bağımsız aday raporu örtüşüyor, veya arkadaşının birinci elden deneyimi
B — Orta	Tek güçlü aday raporu (Medium, Reddit, Glassdoor snippet) veya resmi ama eski süreç dokümanı
C — Düşük / Sentetik	JobMentis benzeri aggregator'lar, AI-özetler, tek kaynaklı Blind yorumları — pratik için faydalı ama "kesin çıktı" sanma

1. Meta-Analiz: GetYourGuide neye önem veriyor?

1.1 Tek cümlelik verdict

GYG, **LeetCode hard fabrikası değil**. Senior backend için sinyal şunu ölçüyor:

"Bu kişi, Java/Spring Boot üretim kodunda bug bulur mu, refactor eder mi, test yazar mı, booking/inventory tarzı domain'de tutarlılık-ölçek trade-off'unu konuşur mu, ve Guiding Principles'a uyumlu ownership gösterir mi?"

1.2 Seviyelere göre ne sorulmuş? (sentez)

Junior / Associate Software Engineer

Gözlem	Kaynak güveni	Kaynak
Live coding: paylaşılan frontend JS/Vue codebase , bug'lar var, feature/fix	B	Reddit r/cscareerquestionsEU — Associate
Odak: mevcut kodu anlama, bug fix, feature ekleme; "beyaz tahta algoritma" vurgusu zayıf	B	Aynı thread + Android challenge pattern
Android için ayrı public challenge: "Berlin tour reviews fetch — happy path bozulmuş app'i pair programming ile düzelt"	A	github.com/getyourguide/android-challenge
Resmi roadmap'te junior için de: phone screen → take-home → CodeLive → technical → pool owner → face-to-face	A (süreci tarif eder, soruları değil)	Top 15 Questions — Tech Roles

Junior'dan çıkarılan ders: GYG junior'da bile "sıfırdan LeetCode"dan çok **mevcut uygulamada çalışabilme** istiyor.

Mid-level Backend Engineer

Gözlem	Kaynak güveni	Kaynak
Codility round geçildikten sonra technical assessment; refactoring vurgusu	B	Blind — Mid-level Backend (liste özeti)
Glassdoor Backend Engineer: Codility sorting → Live coding (refactoring, SOLID, OOP) → motivational → EM behavioral → CTO	B	Glassdoor Backend Engineer UK
Behavioral: past mistakes, feedback to supervisor, non-technical process implemented, proudest project	B	Aynı Glassdoor

Mid'den çıkarılan ders: Algoritma eşiği var (Codility/HackerRank) ama asıl filtre **refactor + SOLID + iletişim**.

Senior Software Engineer (Java / Backend) — senin hedef seviyen

Gözlem	Kaynak güveni	Kaynak
Tipik loop (2025–2026 varyantları): HR → (opsiyonel HackerRank) → Live coding (Spring Boot repo) → System Design → Culture Fit / Behavioral (bazen 3 behavioral)	A/B	Arkadaş notları + Medium — Veenarao, Mar 2026

Live coding: bug fix + feature + refactor + unit test; katmanlar: entity / service / controller / exception handler	A	Medium + Reddit + arkadaş
System design: high-level; Ticketmaster/ticket booking'e yakın; "design ticket service"	A/B	Arkadaş + Medium (Glassdoor referansı)
Java sohbet: GC optimizasyonu, JVM argümanları (interviewer'a bağlı)	A	Arkadaş
Recruiter reach-out'ta HackerRank olmayabiliyor; online apply'da olabiliyor	A	Arkadaş vs Medium
Repo eskiden 1 gün önce geliyordu; şimdi mülakat anında açılıyor deniyor	B	Arkadaş (duyuma dayalı)
AI kullanımı: "sadece doğru kullanıyorsan"; bazı adaylar AI'nin yavaşlattığını söyledi	B	Reddit Java experience
Resmi backend coding interview repo (JDK 21, Spring Boot, Gradle, :8080) — public indekste var, şu an private/404 olabilir	B	github.com/getyourguide/swe-backend-coding-interview (arama indeksi; clone 404)

1.3 Tematik ağırlıklar (ne çalışmalısın?)

Aşağıdaki "önem skoru" kamu raporları + arkadaş notları + GYG'nin kendi engineering blog'larından çıkarılmıştır.

Konu	Senior önem	Junior/Mid önem	Neden?
Java + Spring Boot pratik (debug/refactor/test/endpoint)	★★★★★	★★★★☆	En çok tekrarlanan teknik sinyal
Production mindset (logs, null root-cause, time management)	★★★★★	★★★☆☆	Medium feedback'te fail nedenleri bunlar
System Design: booking / inventory / availability / consistency	★★★★★	★★☆☆☆	Domain'in kalbi; arkadaş Ticketmaster çizmiş
Distributed systems concepts (events, dual-write, source of truth, GraphQL vs replication)	★★★★☆	★★☆☆☆	GYG blog'ları mülakat diliyle aynı
Core Java / JVM / GC	★★★★☆	★★☆☆☆	Arkadaş: sohbet tarzı ama follow-up'lı
SOLID / OOP / clean code	★★★★☆	★★★★☆	Glassdoor live coding
Codility / HackerRank (sorting, use-case program)	★★★☆☆	★★★★☆	Process varyantına bağlı
Classic LeetCode hard / DP / graphs	★★☆☆☆	★★☆☆☆	Dominant sinyal değil (C seviye aggregator'lar abartıyor olabilir)

Behavioral + Guiding Principles (STAR)	★★★★★	★★★★☆	Resmi: hiring'e gömülü; arkadaş: 3 behavioral
Mentorship / roadmap / cross-team	★★★★☆	★☆☆☆☆	Senior JD + growth path

1.4 “Buradan çıkan strateji”

1. **Günde 1 saat Spring Boot live-coding drill** (bug → test → feature → refactor) > 3 saat random LeetCode.
2. **Ticketmaster + Inventory/Availability** system design'ı ezberleme; trade-off konuş.
3. **GC + JVM flags** için sohbet seviyesinde derinlik (allocation, generational, pause vs throughput, common flags).
4. **6 Guiding Principle × 2 STAR hikaye** hazırla.
5. Recruiter'a ilk aramada sor: *HackerRank var mı? Repo ne zaman geliyor? System design var mı? Kaç behavioral?*

2. Süreç haritası (resmi + saha gerçekliği)

2.1 Resmi modern özet (careers — How we hire)

Kaynak: getyourguide.careers/how-we-hire — Güven A

Tech track (yüksek seviye):

1. Application review
2. Call with recruiter
3. Take-home assignment
4. Technical interview
5. Meet your team
6. Final offer

Not: “Process may look slightly different for some roles.”

2.2 Resmi detaylı tech pipeline (2022 blog — hâlâ referans)

Kaynak: [Top 15 Questions](#) — Güven A (süreç isimleri)

Tech Roles:

- Phone screen
- Take-home task
- **CodeLive** tech interview
- Technical screen
- **Pool owner** screen
- Face-to-face

Face-to-face içinde: presentation, stakeholder, team fit, recruiter wrap-up.

2.3 2019 Recruiting 101 (tarihsel ama faydalı)

Kaynak: [Recruiting 101](#) — Güven B (eski)

- Application → Tech Recruiter Zoom → Take-home → Technical (Zoom + Codility/Codeshare)
 - EM Zoom → Onsite (şimdi remote): team exercise / pair / architecture → Team Fit → CTO
 - (bazen) Executive → References → Offer

2.4 Saha varyantları (2025–2026)

Varyant A — Recruiter outreach (arkadaşın, ~Mayıs)

Güven A (birinci el)

Recruiter reach-out

- Technical Interview (Java Spring Boot repo: bug + feature + refactor)
- System Design (high-level, Ticketmaster-benzeri)
- 3× Behavioral (Amazon-benzeri STAR + Guiding Principles)

- HackerRank **yok** (recruiter reach-out olduğu için)
- Repo: 1 gün önce geldi (sonradan “anında veriyorlar” duyumu var)

Varyant B — Online apply Senior Java (Medium, Mar 2026)

Güven B

HR Interview

- HackerRank (Event Registration System)
- Live coding (refactor / unit test / new endpoint; entity→service→controller→exception)
- System design (“design ticket service” Glassdoor’da geçmiş)
- Culture Fit

Feedback örneği (aday ilerleyemedi): root-cause kaçırma, test tamamlayamama, time management, refactor’ta yüzeysel kalma.

Varyant C — Mid Backend (Glassdoor)

Güven B

Recruiter call

- Codility (sorting)
- Live coding (refactor, SOLID, OOP)
- Motivational (senior engineer)
- Behavioral (EM)
- CTO (15 dk sen sor, 15 dk o sorar)

2.5 Senior için “en olası” senaryo (sentez)

Sen Senior'a hazırlanıyorsan, **şu birleşik checklist** en güvenli hazırlık:

#	Aşama	Hazırlık önceliği
1	Recruiter / HR	Motivasyon, stack (Java), process soruları
2	Opsiyonel OA (HackerRank/Codility)	1-2 saatlik use-case Java + sorting/arrays
3	Live coding Spring Boot	En kritik
4	System Design	Booking / ticket / inventory
5	Behavioral × (2-3)	Guiding Principles STAR
6	EM / Pool owner / Culture	Ownership, impact, growth
7	(Nadir) CTO / Exec	Strateji soruları, ne own etmek istediğin

3. Arkadaş notları — birincil kaynak (tam entegrasyon)

Aşağıdaki metin senin paylaştığın konuşmanın yapılandırılmış hali. **Güven A.**

3.1 Süreç (Mayıs civarı, duyuma göre sonra değişmiş olabilir)

- Recruiter reach-out → **HackerRank yok**
- Sıra: **Technical** → **System Design** → **3 Behavioral**
- Tech stack soruları: **sadece Java**; GC sohbet tarzı

3.2 Technical (repo)

- Önceden Java-Spring Boot repo gönderildi
- Mülakatta: **1 bug fix + 1 feature request + 1 refactor**
- Repo timing: **1 gün önce**; duyum: artık mülakat anında

3.3 System Design

- High-level design
- Arkadaşın çizdiği referans: [Hello Interview — Ticketmaster](#)

3.4 Behavioral

- Amazon'a benzer
- STAR
- Guiding Principles** için birkaç hazır örnek

3.5 Java / GC derinliği

- "GC optimizasyonunu nasıl yaparsın?"
- "Hangi JVM argümanlarını kullanabilirsin?"
- Interviewer'a bağlı; sohbet ama follow-up'lı

3.6 Arkadaş notlarından aksiyon listesi

- ☐ Spring Boot mini-projede bug/feature/refactor zamanlı mock (60–75 dk)
 - ☐ Ticketmaster design'ı 45 dk'da whiteboard/Excalidraw'da bitir
 - ☐ GC: allocation → young/old → collectors → flags → prod story
 - ☐ Her Guiding Principle için 1 STAR kartı
-

4. Technical / Live Coding — derinlemesine

4.1 Formatın anatomisi

Kaynaklar: Medium, Reddit, arkadaş, resmi README indeksi.

Ortam

- Zoom + kendi IDE (IntelliJ önerilir)
- Spring Boot uygulama, localhost:8080
- JDK 21 (resmi interview repo requirement — indeks)
- Gradle bootRun
- 2 interviewer: peer + technical manager (Medium)

Görev tipleri (hemen hemen her raporda)

1. **Bug fix** — loglardan / semptomdan root-cause
2. **Feature / new endpoint** — controller → service → repository/entity
3. **Refactor** — logic'i doğru katmana taşı, SOLID
4. **Unit tests** — fix'i kilitle

Katmanlarda bilerek bırakılmış sorunlar (Medium)

- Entity
- Service
- Controller
- Exception handler

4.2 Medium feedback'ten “ne puanlanıyor?”

Kaynak: [Medium guide](#) — **Güven B**

Artılar (isteniyor):

- Kısa, net iletişim; thought process
- Log analizi ile problemleri bulma
- Test yazma isteği
- Takım/stack hakkında merak
- Guidance ile de olsa çözüme varma

Eksiler (eleme sinyali):

- Yanlış null check'e takılıp **root-cause kaçırma** (ör. null supplier)
- Testleri yarım bırakma
- Debug'a fazla süre (ör. 10 dk beklenen iş çok uzuyor) → diğer task'lara zaman kalmıyor
- Refactor'ta sadece "repository'ye taşı" deyip implement etmeme

4.3 Live coding için savaş planı (60–75 dk)

Dakika	Ne yap
0–3	Projeyi ayağa kaldır, endpoint'leri dene, README/domain'i özetle
3–8	Repro: failing request + log + expected vs actual'ı sesli söyle
8–18	Root-cause + minimal fix (önce doğru, sonra güzel)
18–28	Unit test(ler) — happy + null/edge
28–45	Feature / new endpoint iskeleti
45–60	Refactor (SRP, katman, exception handling)
60–70	Edge cases, observability notları, "prod'da ne izlerdim?"
70–75	Sorular sor (team ownership, deploy, on-call)

4.4 Kontrol listesi (mülakat sırasında)

Debug

- ☐ Semptomu yeniden ürettim mi?
- ☐ Stack trace / log satırına gittim mi?
- ☐ "Neden" diye bir seviye daha derine indim mi? (null check'in kendisi değil, null'ın kaynağı)
- ☐ Fix sonrası aynı request'i tekrar denedim mi?

Feature

- ☐ API contract'ı netleştirdim mi? (path, status codes, validation)
- ☐ Service'te business rule, controller'da HTTP mapping
- ☐ Hatalar exception handler'dan mı çıkıyor?

Refactor

- ☐ Tek sorumluluk ihlali var mı?
- ☐ Controller şişmiş mi?
- ☐ Tekrarlayan null/validation var mı?
- ☐ Test edilebilirlik arttı mı?

İletişim

- ☐ Her 2–3 dakikada "şu an X'i yapıyorum çünkü Y"
- ☐ Takılırsam seçenekleri sayıp interviewer'a sor
- ☐ AI kullanıyorsan: çıktıyı doğrula, kör kopyalama yapma

4.5 AI politikası (2025+ saha)

Reddit: recruiter “don’t use it only use it if you think you’re using it right”.

Başka aday: AI’ya açıklılar ama yavaşlattı.

Pratik kural: AI = autocomplete / boilerplate. Architecture, root-cause ve test senin. Interviewer’a “AI ile şunu generate edip doğruluyorum” de.

4.6 Junior vs Senior farkı (aynı format)

Boyut	Junior/Associate	Senior
Codebase	Vue/JS veya basit Spring	Daha kompleks Spring, katmanlı bug’lar
Beklenen	Fix + basit feature	Fix + feature + anlamlı refactor + test
Konuşma	“nasıl düzelttim”	“neden bu design, prod etkisi, alternatif”
Zaman	Guidance daha fazla	Ownership; guidance’a bağımlı kalma

4.7 Alıştırma projeleri (kendi kendine)

1. Mini “tour booking” Spring Boot: Tour , AvailabilitySlot , Booking
2. Bilerek bug bırak: null supplier, wrong status code, N+1, swallowed exception
3. 60 dk timer ile: bug → test → endpoint → refactor
4. İkinci tur: GC flags / heap dump konuşması ekle

5. Online Assessment (HackerRank / Codility) — varsa

5.1 Ne çıkmış?

Görev	Seviye	Güven	Kaynak
Event Registration System (Java use-case)	Senior	B	Medium
Sorting problem (Codility)	Backend / Mid	B	Glassdoor Backend
EventEmitter: on / emit / off	Senior (AmbitionBox/özet)	C-B	AmbitionBox / aggregator özetleri

5.2 Nasıl hazırlan?

- 2–3 saat: Java collections, sorting, hashing, simple OOP design
- “Event registration / booking seat” tarzı küçük domain modelleri
- Edge case + temiz API
- **Hard DP’ye batma** — ROI düşük görünüyor

5.3 Recruiter reach-out ise

Arkadaşın deneyiminde OA yoktu. Yine de 1 akşam ayır; reverse-offer / başka track'te çıkabilir.

6. System Design — Senior'ın asıl savaş alanı

6.1 Bilinen / yüksek olasılıklı prompt'lar

Prompt	Güven	Kaynak
Ticketmaster benzeri high-level booking	A	Arkadaş + Hello Interview linki
"Design ticket service"	B	Medium → Glassdoor referansı
Availability / inventory / isBookable tutarlılığı	A (domain)	GYG microservices redesign
Online migration / dual-write (konuşma derinliği)	A (domain)	BoxOffice Migration

6.2 Ticketmaster skeleton (arkadaşın referansı)

Kaynak: [Hello Interview Ticketmaster](#)

Functional (top 3):

1. View events
2. Search events
3. Book tickets

Non-functional vurgular:

- Search/view → availability öncelikli
- Booking → **consistency** (no double booking)
- Hot event spike (çok kullanıcı, tek event)
- Read-heavy (~100:1)
- Search latency hedefi (ör. <500ms)

Core entities: Event, User, Performer, Venue, Ticket, Booking

API iskeleti:

```
GET /events/:eventId
GET /events/search?...
POST /bookings/:eventId # sonra: reserve + confirm olarak ayır
```

Deep dive konuları (Senior'dan beklenen):

1. Seat hold / reservation TTL
2. Optimistic vs pessimistic locking / inventory counters
3. Idempotency keys (ödeme + book retry)
4. Search: DB filter → Elasticsearch
5. Cache invalidation vs "source of truth"

6. Payment async + booking state machine
7. Waiting room / virtual queue (hot event)

6.3 GYG domain'e çevir (mülakatta fark yaratır)

GYG tour/experience platformu: museum tickets, food tours, supplier integrations, availability & pricing for **200k+ activities**.

Resmi blog'dan mülakat diline çevrilebilir kavramlar:

A) Source of truth vs stale cache

Problem: checkout'ta birden fazla servis kendi cache flag'ine bakıyor → search'te "available", checkout'ta "sold out".

Çözüm dili: tek `isBookable()` sahibi (Inventory), diğerleri okur.

B) Boundary by business capability

Yanlış: "internal vs external supplier" teknik sınırı

Doğru: availability & pricing domain ownership; Connectivity = adapter

C) Explicit events > invisible triggers

`supplier_activated`, `product_deactivated`, `availability_updated`
Observability, testability, replay

D) GraphQL resolve vs replicate

Sık değişmeyen availability için sync read + fallback; analytics için async replicate

E) Migration playbook

Dual-write → flip reads → flip writes → delete old

Feature flags, shadow mode, endpoint-by-endpoint, metrics (4xx/5xx, conversion)

Senior cevap şablonu: "GYG ölçeğinde ben X yapardım çünkü blog'larında da gördüğümüz Y problemi..."

6.4 45-50 dk system design timeline

Dk	İş
0-5	Requirements + NFRs + out-of-scope
5-10	Entities + API
10-25	Happy path architecture (services, DBs, search, cache)
25-40	Deep dive: double-booking + spikepotency + spike spike

40–45	Observability, failure modes, rollout
45–50	Trade-off özeti + sorular

6.5 JobMentis “soruları” — dikkat (Güven C)

[JobMentis GYG SWE](#) travel-domain coding/design prompt’ları listeliyor (k-closest cities, non-overlapping tours, personalized recs, fraud, notifications...). Bunlar **GYG’ye özel doğrulanmış Glassdoor dump’ı gibi durmuyor**; domain-uyumlu pratik için kullan, “kesin sorulur” deme.

Yine de Senior için faydalı pratikler:

- Interval scheduling (tour overlap)
- Geo k-nearest (heap + Haversine)
- Notification system
- Recommendation at scale

7. Java / JVM / GC — sohbet turu checklist

Arkadaş: GC optimization + JVM args. Medium: Core Java + Spring Boot production.

7.1 Bilmen gereken omurga

1. **Allocation first:** çoğu “GC problemi” aslında allocation/churn
2. Generational hypothesis: young vs old
3. Minor / major / mixed (G1)
4. Throughput vs latency collector seçimi
5. Stop-the-world vs concurrent phases (yüksek seviye)
6. Heap sizing: -Xms / -Xmx , region size (G1)
7. Logging: -Xlog:gc* (modern), eski GCUtil logları
8. Tools: JFR, async-profiler, heap dump (Eclipse MAT)
9. Spring tipik allocation kaynakları: JSON mapping, logging, entity graph, unbounded caches

7.2 Sık follow-up’lar (hazır cevap iskeleti)

Q: GC’yi nasıl optimize edersin?

A: Önce ölç (allocation rate, pause time, heap occupancy) → churn’ü azalt (object reuse, doğru data structure, pagination) → sonra collector/flags. “Flag çevirmeden önce kod.”

Q: Hangi JVM argümanları?

Örnek set (ezber değil, gerekçe ile):

- -Xms = -Xmx (prod resize pause riskini azalt)
- -XX:+UseG1GC (genel web default)
- -XX:MaxGCPauseMillis= (hedef, garanti değil)
- -Xlog:gc*:file=gc.log:time,uptime,level,tags
- Container: -XX:+UseContainerSupport , RAM yüzdesi

Q: Memory leak nasıl avlarsın?

Rising old gen / heap dump / dominator tree / classloader veya static map / unbounded cache.

Q: Spring'de sık GC pressure?

Büyük response DTO, N+1, findAll without page, excessive Hibernate dirty checking, chatty microservice JSON.

7.3 Senior barı

Sadece "G1 vardır" deme. Prod hikâyesi hazırla: "p99 pause X ms idi → allocation Y → şöyle düzelttik → metrik Z."

8. Soru bankası (seviyeye göre)

8.1 Junior / Associate — teknik

#	Soru / görev	Kaynak	Güven
J1	Paylaşılan Vue/JS codebase'de bug fix + feature	Reddit Associate	B
J2	Android: Berlin tour reviews — bozuk happy-path app'i pair ile düzelt	android-challenge README	A
J3	"Bu component neden kötü? Nasıl test edersin?"	Challenge değerlendirme kriterleri (platform, architecture, testing)	A

8.2 Mid — teknik + behavioral

#	Soru / görev	Kaynak	Güven
M1	Codility sorting	Glassdoor Backend	B
M2	Live refactor + SOLID + OOP	Glassdoor Backend	B
M3	Past mistakes	Glassdoor	B
M4	Feedback to supervisor + sonrası	Glassdoor	B
M5	Non-technical process you implemented	Glassdoor	B
M6	Most successful / proud project	Glassdoor	B
M7	What you're looking for in next job; success/failure stories	Glassdoor motivational	B

8.3 Senior — teknik

#	Soru / görev	Kaynak	Güven
S1	Spring Boot repo: bug + feature + refactor	Arkadaş	A

S2	Refactor + unit tests + new endpoint; layered bugs	Medium	B
S3	Log-driven debugging; null supplier root-cause	Medium feedback	B
S4	HackerRank Event Registration System	Medium	B
S5	EventEmitter on/emit/off	AmbitionBox özet	C-B
S6	GC optimization + JVM arguments	Arkadaş	A
S7	AI'yi doğru kullanma muhakemesi	Reddit + JD (AI judgment)	B
S8	Code comprehension questions on provided code	AmbitionBox özet	C-B

8.4 Senior — system design

#	Prompt	Kaynak	Güven
D1	Ticketmaster-like booking	Arkadaş	A
D2	Design ticket service	Medium/Glassdoor	B
D3	Availability source-of-truth / isBookable	GYG eng blog	A (domain)
D4	Dual-write migration design	BoxOffice post	A (domain)

8.5 Behavioral (tüm seviyeler; Senior'da daha ağır)

#	Soru	Kaynak	Güven
B1	When is the last time you left your comfort zone?	Aggregated Glassdoor özetleri	B-C
B2	Describe your worst piece of code and why	Aggregated	B-C
B3	Significant technical trade-off	JobMentis / aggregator	C
B4	Relate to one of GYG's principles from past work	Glassdoor (non-eng role'da da geçmiş)	B
B5	How do you collaborate with PM/FE/design in agile?	Aggregated	B-C
B6	Ownership / impact examples (EM interview)	Resmi EM prep blog	A
B7	Most impactful project	Resmi hiring tips	A
B8	Why this role / company?	Resmi hiring tips	A
B9	How have you helped colleagues improve?	Resmi hiring tips	A
B10	What matters most when choosing next company?	Resmi hiring tips	A

8.6 CTO turu (orta/senior bazılarında)

Glassdoor Backend: 15 dk sen soru sor, 15 dk CTO ownership/team situations sorar.

Hazır soru örnekleri (aday tarafından):

- Engineering strategy next 12–18 months?
 - How do mission teams balance platform vs feature work?
 - How is on-call / reliability owned?
 - Where would you want a new senior to create leverage in first 6 months?
-

9. Behavioral & Guiding Principles — Amazon-style STAR paketi

9.1 Resmi Guiding Principles (ezberle)

Kaynak: [Guiding Principles](#) + [CEO post](#) — Güven A

1. **We act customer-first, not me-first**
2. **We use curiosity as our compass**
3. **We aim high and follow through**
4. **We navigate with agility**
5. **We choose growth over comfort**
6. **We build bridges, not islands**

CEO açıkça yazıyor: principles hiring'e gömülü.

9.2 Engineering Principles (teknik behavioral için altın)

Kaynak: [Engineering Principles](#) — Güven A

Build it

- Simple services, clean interfaces
- Clarity over cleverness
- Proven technologies (fast-follower)

Run it

- Detect failures before they matter (tests, canaries)
- Expect and manage failures (graceful degradation)
- Informed operations (SLOs, alerts)

Do it

- Understand the why
- Pragmatism: done > perfect (bilinçli debt)
- Validate with experiments

Improve it

- Leave code better

- Learn from mistakes; share learnings
- Team growth / mentorship

9.3 STAR şablonu (resmi GYG hiring tips)

Kaynak: [40 tips](#)

- Situation → Task → Action (senin aksiyonların) → Result (**metrik**)
- 3-4 hikâyeyi yaz, ezberleme; her birini 2-3 principle'a map et

9.4 Hikâye matrisi (doldur)

Principle	Hikâye başlığı	Metrik	Not
Customer-first			
Curiosity			
Aim high & follow through			
Agility			
Growth over comfort			
Build bridges			

Senior ekstra hikâyeler:

- Mentorship / hiring loop
- Production incident ownership
- Tech debt vs delivery trade-off
- Cross-team migration veya API contract negotiation
- "Wrong boundary" / distributed monolith deneyimi

9.5 Kötü vs iyi cevap

Kötü: "Takım oyuncusuyum, öğrenmeyi severim."

İyi: "Supplier API timeout'larında checkout fail oluyordu (S). Conversion'ı kurtarmak benimdi (T). Circuit breaker + stale-read fallback + alert ekledim, PM ile 'show limited availability' kopyasını yazdık (A). Checkout 5xx %X → %Y, booking conversion +Z (R). Bu customer-first + navigate with agility."

10. Tech stack — JD ve blog'dan gerçek harita

Kaynak: [Greenhouse Senior Backend Supply JD](#) + eng blog'lar — **Güven A**

Sık geçenler

- Java, Spring Boot
- MySQL, PostgreSQL

- Kafka
- Kubernetes
- GraphQL
- Vue.js, Node.js, PHP (legacy migration bağlamı)
- A/B experimentation

JD senior barı (örnek)

- 6+ years
- High proficiency in Java
- Architecture & deployment experience
- Tests
- Mentorship & hiring
- Mature judgment on AI tools

Mülakat sonucu: Frontend bilmek artı; backend senior'da **Java derinliği + design** baskın.

11. Seviye karşılaştırma tablosu (tek bakışta)

Boyut	Junior/Associate	Mid	Senior
OA	Bazen	Sık (Codility)	Process'e bağlı
Live coding	Bug/feature, daha fazla rehberlik	Refactor + SOLID	Bug+feature+refactor+test+prod talk
System design	Hafif / yok	Orta	Zorunlu high-level + concurrency
Java/GC	Temel	Orta	Sohbet + follow-up
Behavioral sayısı	1-2	2-3	2-3 (+ principles map)
Mentorship soruları	Az	Orta	Beklenen
Domain depth (inventory/booking)	Temel ürün bilgisi	Artı	Fark yaratır
AI	Recruiter kuralına uy	Aynı	"Governance ile lead" beklentisi JD'de

12. Blind / Reddit / HN / 4chan — saha notları

12.1 Reddit

- [Java interview experience](#) — Spring bugfix, AI OK
- [Associate FE Vue codebase](#)
- [Data eng](#) — SQL take-home; "competing offer fast-track" deme uyarısı
- [Culture/WLB thread](#) — kültür tartışması; teknik detay az

12.2 Blind

- [SSWE tips isteyen post](#)
- [Interview experience — belirsiz redler / FE vs fullstack beklenti](#)
- Company posts: Mid-level'da Codility → refactor technical

Ders: Process uzun hissedilebiliyor; feedback gecikebiliyor; rol tanımı (FE-focus fullstack) netleştirebilir.

12.3 Hacker News

GYG için **spesifik mülakat walkthrough bulunamadı** (fonlama / Zurich ekosistemi bahisleri var). Örnek alakasız ama aranan: [GYG Series E HN](#).

12.4 4chan / diğer

Anlamlı, doğrulanabilir GYG SWE mülakat dump'ı **bulunamadı**.

12.5 Glassdoor

Doğrudan sayfalar login-wall / 401; yine de indekslenmiş içerik:

- [Ana interview hub](#) — Senior SWE ~15 review
- [Backend Engineer UK](#)
- Genel: difficulty ~3.0–3.8 bandı (kaynaklar arası değişir); eng interviews “pratik”

Aksiyon: Sen login'li Glassdoor'dan Senior SWE + Backend filtrelerini oku; bu guide'daki B/C maddeleri güncelle.

13. 14 günlük hazırlık planı (Senior)

Hafta 1 — Technical + Java

Gün	Odak
1	Spring Boot tour-booking skeleton; controller/service/repo
2	2× timed bug-fix mock (45 dk) + test
3	Feature endpoint + exception handler + validation
4	Refactor drill (God service → clean)
5	GC/JVM deep dive + 1 prod story yaz
6	Full 75 dk mock (bug+feature+refactor+test) kaydet
7	Zayıf noktaları kapat; Spring Data / transactions tekrarı

Hafta 2 — Design + Behavioral + Polish

Gün	Odak
8	Ticketmaster end-to-end (Hello Interview)
9	GYG twist: isBookable + supplier adapter + events
10	Deep dive only: double-booking + idempotency + hot event
11	Guiding Principles STAR kartları (6×)
12	Engineering Principles'ı design cevaplarına yedir
13	Full loop simulation: 75 coding + 45 design + 45 behavioral
14	Recruiter soruları, CTO soruları, dinlen

14. Kaynaklar (link listesi)

Resmi GetYourGuide

1. [How we hire](#)
2. [Recruiting 101 \(2019\)](#)
3. [Top 15 candidate questions \(process\)](#)
4. [Your Interview \(engineering prep tips\)](#)
5. [Guiding Principles](#)
6. [Guiding Principles — CEO post](#)
7. [Engineering Principles](#)
8. [Growth path for engineers](#)
9. [40 hiring tips / STAR](#)
10. [Microservices redesign / isBookable](#)
11. [BoxOffice migration](#)
12. [Customer-obsessed Inventory eng](#)
13. [Collaboration Driven Development / PHP → Java](#)
14. [Senior Backend JD \(Greenhouse örnek\)](#)

Aday deneyimleri

15. [Medium — Senior Java Berlin \(Mar 2026\)](#)
16. [Reddit — Java Spring live coding + AI](#)
17. [Reddit — Associate Vue codebase](#)
18. [Blind — SSWE coding tips](#)
19. [Blind — Interview experience thread](#)
20. [Blind company posts index](#)

Glassdoor / aggregator

21. [Glassdoor GYG interviews hub](#)
22. [Glassdoor Backend Engineer \(UK\)](#)
23. [AmbitionBox interviews](#)

24. [JobMentis SWE \(Güven C — pratik için\)](#)

System design & coding artifacts

25. [Hello Interview — Ticketmaster](#)

26. [GYG swe-be-coding-interview \(indeks;_private olabilir\)](#)

27. [GYG android-challenge](#)

Video / kültür

28. [YouTube — 3 Myths about Senior SWEs at GYG](#)

Bu guide'ın birincil özel kaynağı

29. **Arkadaşın Mayıs süreci notları** (bu repo dokümanının §3'ü) — HackerRank yok; Technical → SD → 3 Behavioral; Spring repo; GC sohbet; Ticketmaster-benzeri SD.

15. Recruiter'a sorulacak sorular (ilk arama)

1. Bu rol için güncel aşamalar neler? (OA var mı?)
 2. Live coding repo ne zaman paylaşıyor?
 3. System design high-level mı, low-level mı?
 4. Behavioral sayısı ve Guiding Principles odaklı mı?
 5. AI tool kullanım politikası nedir?
 6. Stack beklentisi: sadece Java mı, Vue/PHP da mı?
 7. Pool / mission team belli mi?
 8. Timeline ve feedback SLA?
-

16. Mülakat günü tek sayfalık cheat sheet

Technical

1. Repro → logs → root-cause → minimal fix → test → feature → refactor
2. Sesli düşün; zamanı yönet; yarım refactor > hiç test yok
3. Null'un kendisini değil kaynağını bul

System Design

1. FR/NFR netleştir (booking = consistency)
2. Entities + API
3. Reserve/hold + confirm
4. Hot event + locking + idempotency
5. GYG dili: source of truth, explicit events, graceful degradation, SLOs

Behavioral

1. STAR + metrik
2. Principle ismini açıkça bağla
3. Failure + öğrenme anlat

Java/GC

1. Measure → reduce allocation → then tune
 2. 3-4 flag + gerekçe
 3. Bir prod story
-

17. Bilinçli boşluklar / uyarılar

1. **Süreç değişiyor** — arkadaşın Mayıs deneyimi ile Medium 2026 ve resmi 2019/2022 dokümanları birebir aynı değil. Recruiter gerçeği kesinleştirir.
 2. Glassdoor tam metinleri bu ortamda login-wall yüzünden kısmen snippet ile alındı; sen hesabınla taze oku.
 3. JobMentis soruları büyük olasılıkla **domain-uyarlanmış sentetik** — C güven.
 4. swe-be-coding-interview public indekste görünüyor ama clone 404; recruiter paylaşıncı çalıştır.
 5. HN/4chan'de işe yarar GYG SWE dump'ı yoktu.
 6. Blind'de kültür/red şikayetleri var; teknik hazırlığı engellemesin ama rol scope'unu net sor.
-

18. Son söz — “sadece bu guide ile” minimum bar

Eğer zamanın kısıtlıysa **yalnızca şunları** bitir:

1. 3× timed Spring Boot live-coding mock
2. Ticketmaster + GYG isBookable twist design
3. GC/JVM 1 sayfalık not + 1 hikâye
4. 6 Guiding Principle STAR kartı
5. Engineering Principles'ı ezberle (clarity, proven tech, SLOs, done>perfect, leave better)

Bu beşli, kamu kaynakları + arkadaş notlarının kesişim kümesi.

İyi şanslar — customer-first ve curiosity ile gir; coding turunda log'u oku, design'da double-booking'i kapat, behavioral'da metrik söyle.

Doküman bakımı: Glassdoor'dan yeni Senior SWE review'ları geldikçe §8 tablosuna satır ekle; güven seviyesini A/B/C ile işaretle.